Les branches avec Git - Rebaser

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Solution pour fusionner 2 branches **4**

A. Solution pour fusionner 2 branches 4

B. Exercice : Quiz 6

III - Git merge vs git rebase **8**

A. Git merge vs git rebase 8

B. Exercice : Quiz 10

IV - Essentiel **12**

V - Auto-évaluation **13**

A. Exercice 13

B. Test 13

Solutions des exercices **15**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

- **Durée : 1 h**
- **Environnement de travail :** PC ou Mac avec 8 Go de RAM minimum
- **Prérequis :** notions de Git

Contexte

Les branches permettent de travailler sur différentes fonctionnalités ou corrections de bugs en parallèle sans perturber votre travail principal. Cela vous permet de vous concentrer sur une seule tâche à la fois et de garder une trace claire de chaque modification apportée à votre code.

Dans un projet Git, il est conseillé de créer des branches qui correspondent à des issues afin de faciliter la gestion du développement de l'application. En effet, chaque issue représente une tâche spécifique à accomplir dans le projet, comme une fonctionnalité à implémenter ou un bug à corriger.

Lorsque vous travaillez sur un projet avec Git, vous pouvez donc créer plusieurs branches pour travailler sur différentes fonctionnalités ou corrections de bugs en parallèle. Cela permet de séparer les modifications en cours de développement de la branche principale. D'ailleurs, celle-ci est très souvent appelée la branche « *master* ». Cependant, à un moment donné, vous devrez fusionner vos modifications de la branche de développement vers la branche master.

Il est possible de réaliser cette opération à travers le **git merge** et le **git rebase**. Il est important de comprendre les avantages et les inconvénients de chaque méthode pour choisir la meilleure option en fonction de votre projet.

À travers ce cours, nous allons essayer de comprendre la particularité de la fusion de branche en découvrant le git merge, le git rebase et en les comparant.

Tous droits réservés à STUDI - Reproduction interdite

II Solution pour fusionner 2 branches

A. Solution pour fusionner 2 branches

Git merge

Le git merge est une opération qui permet de fusionner 2 ou plusieurs branches en une seule, en créant un nouveau **commit** qui contient les modifications des branches fusionnées.

Cela peut être utile lorsque vous avez terminé le développement d'une fonctionnalité ou d'une correction de bug sur une branche de développement et que vous souhaitez l'intégrer à la branche principale.

Méthode

Prenons l'exemple d'un projet web simple qui comporte une branche « *master* » et une branche « *dev* ». Imaginons que vous souhaitez ajouter une nouvelle fonctionnalité à votre application, et vous travaillez sur la branche « *dev* ». Une fois que votre travail sera terminé, il faudra fusionner la branche « *dev* » avec la branche « *master* » afin d'intégrer toutes vos modifications à la branche principale.

Pour réaliser cette opération, vous pouvez utiliser la commande `git merge` en fournissant le nom de la branche à fusionner.

Tout d'abord, vous devez vous assurer d'être sur la branche « *master* ». Pour ce faire, vous exécutez la commande `git checkout master`. Ensuite, pour fusionner la branche « *dev* » dans la branche « *master* », vous exécutez la commande `git merge dev` :

```
$ git checkout master
Switched to branch 'master'
$ git merge dev
Updating abcd123..efgh456
Fast-forward
 src/index.html  |  3 ++-
 src/style.css   |  5 ++++-
 2 files changed, 6 insertions(+), 2 deletions(-)
```

La commande `Git merge` détecte automatiquement les modifications de la branche « *dev* » et crée un nouveau commit qui intègre ces modifications à la branche « *master* ». Dans cet exemple, Git a effectué une fusion rapide (*Fast-forward*) car il n'y avait pas de conflits à résoudre entre les 2 branches.

Si Git détecte des conflits de fusion, il affiche un message vous indiquant les fichiers concernés et les sections conflictuelles.

Par exemple, si vous aviez modifié le même fichier sur la branche « *master* » et la branche « *dev* », vous pourriez obtenir un message d'erreur similaire à celui-ci :

```
$ git merge dev
Auto-merging src/index.html
CONFLICT (content): Merge conflict in src/index.html
Automatic merge failed; fix conflicts and then commit the result.
```

Dans ce cas, vous devez résoudre manuellement les conflits en modifiant les fichiers concernés et en marquant les conflits comme résolus à l'aide de la commande `git add -A`. Une fois que vous avez résolu les conflits, vous pouvez terminer la fusion en exécutant la commande `git merge --continue`.

Tous droits réservés à STUDI - Reproduction interdite

Le git merge est donc une opération simple et pratique qui permet de fusionner 2 ou plusieurs branches en une seule. Il est utile lorsque vous avez terminé le développement d'une fonctionnalité ou d'une correction de bug sur une branche de développement et que vous souhaitez l'intégrer à la branche principale.

Git add -A

La commande `git add -A` est une commande Git qui permet d'ajouter tous les fichiers modifiés, supprimés et nouvellement créés à l'**index Git**. L'index Git est un espace de travail intermédiaire qui sert à préparer les fichiers qui seront inclus dans le prochain commit.

Méthode

Voici comment utiliser la commande `git add -A` :

1. Ouvrez votre terminal ou votre interface de ligne de commande (CLI).
2. Accédez au répertoire de votre projet Git en utilisant la commande `cd`.
3. Utilisez la commande `git add -A` pour ajouter tous les fichiers modifiés, supprimés et nouvellement créés à l'index Git.
4. Utilisez la commande `git status` pour vérifier que les fichiers ont bien été ajoutés à l'index.
5. Créez un nouveau commit en utilisant la commande `git commit -m 'votre message de commit'` ou effectuez la commande `git merge –continue` (si vous êtes en train de fusionner 2 branches).

```
$ git add -A
$ git status
$ git commit -m "mon commit"
```

La commande `git add -A` est utile lorsque vous avez effectué des modifications dans plusieurs fichiers de votre projet et que vous souhaitez tous les inclure dans le prochain commit. Elle peut également être utile si vous avez supprimé des fichiers ou si vous avez créé de nouveaux fichiers dans votre projet.

Il est important de noter que la commande `git add -A` ajoute tous les fichiers modifiés, supprimés et nouvellement créés à l'index Git, même ceux qui ont été modifiés en dehors de Git. Il est donc important de vérifier les fichiers qui ont été ajoutés à l'index en utilisant la commande `git status` avant de créer un nouveau commit.

Git rebase

Le git rebase est une opération qui permet de fusionner des branches, mais en réécrivant l'historique de développement de manière linéaire.

Cette opération déplace tous les commits de la branche à fusionner en amont de la branche cible, ce qui crée un historique de développement plus facile à comprendre et à gérer. Cependant, legGit rebase peut être plus complexe à utiliser et peut provoquer des conflits lorsqu'il réécrit l'historique de développement.

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Prenons l'exemple d'un projet web simple qui comporte 2 branches, la branche « *main* » et la branche « *new-feature* ». Vous travaillez sur la branche « *new-feature* » pour ajouter une nouvelle fonctionnalité à votre application. Lorsque vous avez terminé votre travail, vous devez fusionner votre branche « *new-feature* » avec la branche « *main* » pour intégrer vos modifications à la branche principale.

Pour réaliser cette opération à l'aide du git rebase, vous devez d'abord vous assurer d'être sur la branche « *new-feature* » en exécutant la commande `git switch new-feature`. Ensuite, vous pouvez réécrire la branche « *new-feature* » à partir de la branche « *main* » en exécutant la commande `git rebase main` :

```
$ git checkout new-feature
Switched to branch 'new-feature'
$ git rebase main
First, rewinding head to replay your work on top of it...
Applying: Added new feature
```

La commande `git rebase` réécrit l'historique de développement de la branche « *new-feature* » en plaçant tous les commits de la branche « *new-feature* » en amont de la branche « *main* ». Dans cet exemple, le seul commit de la branche « *new-feature* » est « *Added new feature* », et il est appliqué à la branche « main ». Si Git détecte des conflits de fusion lors du rééquilibrage, il vous invitera à les résoudre manuellement de la même manière que pour un git merge.

Il est important de noter que le git rebase peut modifier l'historique de développement, ce qui peut causer des problèmes si d'autres développeurs travaillent sur la même branche. Si vous utilisez le git rebase, il est donc recommandé de le faire sur une branche de développement locale plutôt que sur une branche partagée.

Le git rebase est une opération puissante qui permet de réécrire l'historique de développement de manière linéaire. Il peut être plus complexe à utiliser que le git merge et peut provoquer des conflits lorsqu'il réécrit l'historique de développement. Cependant, il peut être utile dans certaines situations pour créer un historique de développement plus facile à comprendre et à gérer.

B. Exercice : Quiz

[solution n°1 p. 15]

Question 1

Qu'est-ce que la fusion de branches avec git merge ?

- Une opération qui permet de fusionner 2 ou plusieurs branches en une seule
- Une opération qui permet de cloner une branche existante pour la fusionner avec une autre branche
- Une opération qui permet de supprimer une branche de développement

Question 2

Que se passe-t-il si Git détecte des conflits de fusion lors de l'opération git merge ?

- Git crée automatiquement un nouveau commit qui résout les conflits

Tous droits réservés à STUDI - Reproduction interdite

Git affiche un message vous indiquant les fichiers concernés et les sections conflictuelles

Git annule la fusion et ne crée pas de nouveau commit

Question 3

Comment peut-on fusionner une branche de développement avec la branche principale à l'aide de git merge ?

En exécutant la commande « *git merge nom_de_la_branche_de_développement* » en étant sur la branche principale

En exécutant la commande « *git merge nom_de_la_branche_principale* » en étant sur la branche de développement

En exécutant la commande « *git merge nom_de_la_branche_de_développement* » en étant sur la branche de développement

Question 4

Qu'est-ce que la commande « *git add -A* » ?

Une commande qui permet de supprimer des fichiers de l'index Git

Une commande qui permet d'ajouter tous les fichiers modifiés, supprimés et nouvellement créés à l'index Git

Une commande qui permet de créer une nouvelle branche Git

Question 5

Qu'est-ce que le git rebase ?

Une opération qui permet de supprimer des branches inutiles

Une opération qui permet de fusionner 2 ou plusieurs branches en une seule

Une opération qui permet de réécrire l'historique de développement de manière linéaire

Tous droits réservés à STUDI - Reproduction interdite

III Git merge vs git rebase

A. Git merge vs git rebase

Rappel

Git merge et git rebase sont donc 2 commandes permettant de fusionner des branches. Cependant, leur fonctionnement est différent et chacune a des avantages et des inconvénients.

Git merge

Méthode

La commande `git merge` crée un nouveau commit qui combine l'historique de la branche fusionnée avec l'historique de la branche de destination. Voici un exemple d'utilisation de `git merge` :

```
$ git switch main
$ git merge feature
```

Dans cet exemple, nous sommes sur la branche « *main* » et nous fusionnons la branche « *feature* ». Git va créer un nouveau commit qui représente la fusion des 2 branches. Le résultat dans Git Graph sera similaire à ceci :

```
*   Merge branch 'feature' into main
|\
| * Add new feature
| * Update file
|/
* Update README.md
```

Git rebase

Méthode

La commande `git rebase` permet également de fusionner des branches, mais elle le fait en replaçant les commits de la branche fusionnée au-dessus de la branche de destination. Cela permet d'obtenir un historique de commits plus linéaire. Voici un exemple d'utilisation de `git rebase` :

```
$ git switch feature
$ git rebase main
```

Dans cet exemple, nous sommes sur la branche « *feature* » et nous fusionnons la branche « *main* ». Git va déplacer les commits de « *feature* » sur la branche « *main* ». Le résultat dans Git Graph sera similaire à ceci :

```
* Add new feature
* Update file
* Update README.md
```

La principale différence entre `git merge` et `git rebase` est la façon dont les commits sont fusionnés. `git merge` crée un nouveau commit de fusion, tandis que `git rebase` déplace les commits existants.

Tous droits réservés à STUDI - Reproduction interdite

L'avantage de `git merge` est qu'il préserve l'historique de la branche fusionnée, tandis que l'avantage de `git rebase` est qu'il permet d'obtenir un historique plus linéaire. De l'autre côté, l'inconvénient de `git merge` est que l'historique peut devenir plus complexe, tandis que l'inconvénient de `git rebase` est qu'il peut modifier l'historique existant et rendre la fusion plus difficile à comprendre pour les autres développeurs.

Il est important de choisir la bonne commande en fonction de la situation. Par exemple, si vous travaillez sur une branche locale et que vous voulez mettre à jour votre code avec les dernières modifications de la branche principale, `git merge` est probablement la meilleure option. Si vous travaillez sur une branche partagée avec d'autres développeurs, `git rebase` peut être une meilleure option pour éviter un historique trop complexe.

En utilisant Git Graph ou d'autres outils similaires, vous pouvez facilement visualiser les différences entre `git merge` et `git rebase` et choisir la meilleure option pour votre situation.

Voici la commande qui vous permettra d'utiliser directement Git Graph dans le terminal :

```
$ git config --global alias.graph "log --all --graph --decorate --oneline"
```

Le Secure Hash Algorithm (SHA)

Le *Secure Hash Algorithm* (SHA) d'une branche Git est l'**identifiant unique** qui est attribué à la branche dans le système de contrôle de version Git. Plus précisément, le SHA est une chaîne de caractères hexadécimaux de 40 caractères qui permet d'identifier de manière unique chaque commit, branche, étiquette et arborescence de Git.

Chaque fois qu'un commit est créé sur une branche Git, un nouveau SHA est généré pour ce commit, qui est ensuite utilisé pour identifier ce commit de manière unique. Lorsque des modifications sont apportées à la branche, le SHA de la branche change pour refléter les nouveaux commits ajoutés à la branche.

Le SHA d'une branche Git peut être utilisé pour identifier de manière unique une branche et pour effectuer diverses opérations, telles que la fusion de branches ou la création de nouvelles branches à partir d'une branche existante. Pour trouver le SHA d'une branche Git, vous pouvez utiliser la commande `git rev-parse` suivie du nom de la branche.

Méthode

Voici un exemple d'utilisation de SHA pour une fusion de branche en utilisant le terminal.

Tout d'abord, nous allons créer une nouvelle branche « *feature* » et y ajouter quelques commits afin que l'exemple soit concret :

```
$ git checkout -b feature
$ echo "Commit 1" >> file.txt
$ git add file.txt
$ git commit -m "Added commit 1"
$ echo "Commit 2" >> file.txt
$ git add file.txt
$ git commit -m "Added commit 2"
$ echo "Commit 3" >> file.txt
$ git add file.txt
$ git commit -m "Added commit 3"
```

Ensuite, nous allons basculer sur la branche principale et y ajouter un nouveau commit :

```
$ git checkout main
$ echo "Commit 4" >> file.txt
$ git add file.txt
$ git commit -m "Added commit 4"
```

Tous droits réservés à STUDI - Reproduction interdite

Maintenant, nous allons fusionner la branche « *feature* » dans la branche principale en utilisant le SHA du commit sur la branche « *feature* » :

```
$ git merge c1f0d56
```

Ici, « *c1f0d56* » est le SHA du dernier commit sur la branche « *feature* ».

Git va maintenant créer un nouveau commit de fusion qui combine les modifications apportées sur les 2 branches :

```
$ Merge branch 'feature'
```

Pour vérifier que tout s'est bien passé, nous pouvons utiliser la commande git log pour afficher l'historique des commits :

```
$ git log --oneline --graph --decorate
```

Le résultat affiché devrait ressembler à ceci :

```
*   e15602b (HEAD -> main) Merge branch 'feature'
|\
| * c1f0d56 (feature) Added commit 3
| * 2a6e7d2 Added commit 2
| * 2c19483 Added commit 1
|/
* 729b872 Added commit 4
```

Comme vous pouvez le voir, la branche « *feature* » a été fusionnée avec succès dans la branche principale en utilisant le SHA du dernier commit sur la branche « *feature* ».

B. Exercice : Quiz

[solution n°2 p. 16]

Question 1

Quel est l'avantage de git merge ?

- Il permet d'obtenir un historique plus linéaire
- Il préserve l'historique de la branche fusionnée
- Il modifie l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

Question 2

Quel est l'inconvénient de git merge ?

- Il préserve l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs
- Il peut modifier l'historique existant et rendre la fusion plus difficile à comprendre pour les autres développeurs
- L'historique peut devenir plus complexe

Question 3

Quel est l'avantage de git rebase ?

Tous droits réservés à STUDI - Reproduction interdite

Il préserve l'historique de la branche fusionnée

Il permet d'obtenir un historique plus linéaire

Il modifie l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

Question 4

Quel est l'inconvénient de git rebase ?

Il peut modifier l'historique existant et rendre la fusion plus difficile à comprendre pour les autres développeurs

Il préserve l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

Il ne présente pas d'inconvénient

Question 5

Quelle est la différence entre git merge et git rebase ?

git merge crée un nouveau commit de fusion, tandis que git rebase déplace les commits existants

git merge déplace les commits existants, tandis que git rebase crée un nouveau commit de fusion

Il n'y a pas de différence entre git merge et git rebase

Tous droits réservés à STUDI - Reproduction interdite

IV Essentiel

La fusion de branches est une fonctionnalité essentielle de Git qui permet aux développeurs de travailler efficacement sur des projets collaboratifs. Grâce à Git, il est possible de travailler simultanément sur des branches différentes sans compromettre la stabilité du code.

Dans la première partie de ce cours, nous avons abordé les concepts de base de la fusion de branches avec Git, avec les méthodes git merge et git rebase, et vu comment utiliser ces commandes.

Dans la deuxième partie, nous avons examiné la différence entre git merge et git rebase. Nous avons vu que git merge crée un nouveau commit de fusion, tandis que git rebase déplace les commits existants sur la branche de destination. Nous avons également vu les avantages et les inconvénients de chaque méthode et comment choisir la meilleure méthode en fonction de la situation.

Enfin, nous avons brièvement examiné le concept de SHA, qui est un identifiant unique attribué à chaque commit, branche, étiquette et arborescence de Git. Nous avons vu comment utiliser le SHA pour identifier de manière unique une branche et effectuer diverses opérations telles que la fusion de branches ou la création de nouvelles branches à partir d'une branche existante.

En résumé, Git est un outil extrêmement puissant et flexible pour la gestion des versions de code. La fusion de branches est l'une de ses fonctionnalités clés, permettant aux développeurs de travailler efficacement sur des projets collaboratifs. En comprenant les concepts de base de Git et en choisissant judicieusement entre les méthodes de fusion de branches, les développeurs peuvent travailler en toute confiance sur des projets complexes et collaboratifs.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous avez récemment rejoint une société de publicité en tant que développeur web junior et on vous a affecté à un projet existant. Le projet est déjà en cours depuis un certain temps et comporte plusieurs fonctionnalités et branches. Vous avez cloné le projet à partir du référentiel distant et vous êtes maintenant prêt à commencer à travailler dessus.

Question 1

[solution n°3 p. 18]

Vous travaillez sur ce projet à 2 et vous devez ajouter une nouvelle fonctionnalité. On vous demande de créer une nouvelle branche nommée « *feature* » pour y travailler et d'effectuer au moins 3 commits. Entre chaque commit, vous devez faire des modifications sur la branche principale « *master* ». Une fois que vous avez terminé, vous devez fusionner la branche « *feature* » sur la branche « *master* ». Pour cela, vous devez simuler le travail Git en utilisant les commandes nécessaires dans l'ordre chronologique. Enfin, vous devez utiliser Git Graph ou un outil similaire pour fournir le graphique des branches.

Vous êtes un développeur web junior travaillant sur un projet existant pour une entreprise de commerce électronique. Vous travaillez en collaboration avec un autre développeur senior sur le projet. Le projet contient plusieurs branches et fonctionnalités qui ont été développées au fil du temps. Vous avez cloné le projet à partir du référentiel distant et vous êtes prêt à y travailler.

Question 2

[solution n°4 p. 19]

Vous devez ajouter une nouvelle fonctionnalité au projet en utilisant une nouvelle branche nommée « *feature2* ». Cependant, il y a eu des modifications sur la branche principale « *master* » depuis que vous avez cloné le référentiel. Vous devez donc mettre à jour votre branche « *feature2* » pour inclure ces modifications.

Ensuite, vous devez effectuer au moins 3 commits sur la branche « *feature* » et faire des modifications sur la branche « *master* » entre chaque commit. Une fois que vous avez terminé, vous devez fusionner la branche « *feature* » sur la branche « *master* » en utilisant git rebase. Pour cela, vous devez simuler le travail Git en utilisant les commandes nécessaires dans l'ordre chronologique.

Enfin, vous devez utiliser Git Graph ou un outil similaire pour fournir le graphique des branches.

B. Test

Exercice 1 : Quiz

[solution n°5 p. 20]

Question 1

Comment réaliser une opération de git merge ?

- Avec la commande « *git clone* »
- Avec la commande « *git merge* »
- Avec la commande « *git add -A* »

Question 2

Quelle commande convient le mieux si vous travaillez sur une branche locale et que vous voulez mettre à jour votre code avec les dernières modifications de la branche principale ?

Tous droits réservés à STUDI - Reproduction interdite

- Git merge
- Git rebase
- Les deux

Question 3

Quelle commande convient le mieux si vous travaillez sur une branche partagée avec d'autres développeurs pour éviter un historique trop complexe ?

- Git merge
- Git rebase
- Les deux

Question 4

Que représente le SHA dans Git ?

- L'identifiant unique d'une branche Git
- L'identifiant unique d'un commit, d'une branche, d'une étiquette et d'une arborescence de Git
- L'historique des commits dans Git

Question 5

Comment peut-on trouver le SHA d'une branche Git ?

- En utilisant la commande « *git log* »
- En utilisant la commande « *git rev-parse* » suivie du nom de la branche
- En utilisant la commande « *git branch* »

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 6]

Question 1

Qu'est-ce que la fusion de branches avec git merge ?

- ✓ Une opération qui permet de fusionner 2 ou plusieurs branches en une seule
- Une opération qui permet de cloner une branche existante pour la fusionner avec une autre branche
- Une opération qui permet de supprimer une branche de développement

La fusion de branches avec git merge permet non seulement la fusion de branches, mais crée également un nouveau commit.

Question 2

Que se passe-t-il si Git détecte des conflits de fusion lors de l'opération git merge ?

- Git crée automatiquement un nouveau commit qui résout les conflits
- ✓ Git affiche un message vous indiquant les fichiers concernés et les sections conflictuelles
- Git annule la fusion et ne crée pas de nouveau commit

Si Git détecte des conflits de fusion lors de l'opération git merge, il affiche un message vous indiquant les fichiers concernés et les sections conflictuelles. Vous devez alors résoudre manuellement ces conflits en modifiant les fichiers qui posent problème.

Question 3

Comment peut-on fusionner une branche de développement avec la branche principale à l'aide de git merge ?

- ✓ En exécutant la commande « *git merge nom_de_la_branche_de_développement* » en étant sur la branche principale
- En exécutant la commande « *git merge nom_de_la_branche_principale* » en étant sur la branche de développement

Tous droits réservés à STUDI - Reproduction interdite

En exécutant la commande « *git merge nom_de_la_branche_de_développement* » en étant sur la branche de développement

Pour fusionner une branche avec git merge, il faut exécuter la commande à partir de la branche principale. Il vous faudra peut-être switcher de branche avant d'effectuer l'opération.

Question 4

Qu'est-ce que la commande « *git add -A* » ?

Une commande qui permet de supprimer des fichiers de l'index Git

✓ Une commande qui permet d'ajouter tous les fichiers modifiés, supprimés et nouvellement créés à l'index Git

Une commande qui permet de créer une nouvelle branche Git

« *git add –A* » est une manière rapide et pratique d'indiquer à Git que vous voulez inclure toutes les modifications que vous avez apportées dans le prochain commit.

Question 5

Qu'est-ce que le git rebase ?

Une opération qui permet de supprimer des branches inutiles

Une opération qui permet de fusionner 2 ou plusieurs branches en une seule

✓ Une opération qui permet de réécrire l'historique de développement de manière linéaire

Le git rebase est une opération qui permet de réécrire l'historique de développement de manière linéaire en déplaçant tous les commits de la branche à fusionner en amont de la branche cible.

Solution n°2

[exercice p. 10]

Question 1

Quel est l'avantage de git merge ?

Il permet d'obtenir un historique plus linéaire

Tous droits réservés à STUDI - Reproduction interdite

✓ Il préserve l'historique de la branche fusionnée

Il modifie l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

L'avantage qu'offre git merge est qu'il préserve l'historique de la branche fusionnée. Cela signifie que vous pouvez facilement voir les modifications apportées à la branche fusionnée et qui a effectué ces modifications.

Question 2

Quel est l'inconvénient de git merge ?

Il préserve l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

Il peut modifier l'historique existant et rendre la fusion plus difficile à comprendre pour les autres développeurs

✓ L'historique peut devenir plus complexe

L'inconvénient de git merge est que l'historique peut malheureusement devenir plus complexe. Cela peut rendre la compréhension de l'historique des modifications apportées à la branche fusionnée plus difficile.

Question 3

Quel est l'avantage de git rebase ?

Il préserve l'historique de la branche fusionnée

✓ Il permet d'obtenir un historique plus linéaire

Il modifie l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs

L'avantage qu'offre git rebase est qu'il permet d'obtenir un historique de commits plus linéaire. Cela peut rendre l'historique des modifications plus facile à comprendre.

Question 4

Quel est l'inconvénient de git rebase ?

Tous droits réservés à STUDI - Reproduction interdite

- ✓ Il peut modifier l'historique existant et rendre la fusion plus difficile à comprendre pour les autres développeurs
- Il préserve l'historique existant et rend la fusion plus difficile à comprendre pour les autres développeurs
- Il ne présente pas d'inconvénient

Git rebase peut modifier l'historique existant en déplaçant les commits de la branche fusionnée et les replaçant sur la branche de destination. Cela peut rendre la compréhension de l'historique plus difficile pour les autres développeurs qui n'ont pas été impliqués dans la fusion. C'est son inconvénient principal.

Question 5

Quelle est la différence entre git merge et git rebase ?

- ✓ git merge crée un nouveau commit de fusion, tandis que git rebase déplace les commits existants
- git merge déplace les commits existants, tandis que git rebase crée un nouveau commit de fusion
- Il n'y a pas de différence entre git merge et git rebase

Les commandes git merge et git rebase ont un fonctionnement différent. Git merge crée un nouveau commit de fusion qui combine les historiques de 2 branches, tandis que git rebase déplace les commits de la branche fusionnée au-dessus de la branche de destination.

[exercice p. 13]

Solution n°3

```
1. Création de la branche feature et bascule dessus
git switch -c feature

2. Première modification sur feature
git add .
git commit -m "Ajout de la nouvelle fonctionnalité"

3. Retour sur master
git switch master

4. Modification de master
git add .
git commit -m "Modification de la branche principale"

5. Retour sur feature
git switch feature
```


Tous droits réservés à STUDI - Reproduction interdite

```
6. Deuxième modification sur feature
git add .
git commit -m "Amélioration de la fonctionnalité"

7. Retour sur master
git switch master

8. Nouvelle modification de master
git add .
git commit -m "Nouvelle modification de la branche principale"

9. Retour sur feature
git switch feature

10. Troisième modification sur feature
git add .
git commit -m "Finalisation de la fonctionnalité"

11. Retour sur master
git switch master

12. Fusion de feature dans master
git merge feature

13. En cas de conflit :

résolution manuelle des fichiers concernés
git add .
git commit -m "Résolution des conflits avant fusion"
```

Pour visualiser le graphique des branches, vous pouvez utiliser la commande `git log --oneline --graph --decorate` ou utiliser un outil comme Git Graph.

```
* d1a8f30 (HEAD -> master) Fix avant merge  'feature'
|\
| * e32a9a9 (feature) Ajout d'un troisième commit à la branche feature
| * 39bca92 Ajout d'un deuxième commit à la branche feature
| * 05b8ca1 Ajout de la nouvelle fonctionnalité
|/
* 35077e4 Nouvelle modification de la branche principale
* 4f5202d Modification de la branche principale
* 0c1ab11 Initial commit
```

[exercice p. 13]

Solution n°4

```
//Création de la branche feature et bascule sur cette branche
$ git switch -c feature2

//Ajout de la nouvelle fonctionnalité
$ git add .
$git commit -m "Commit 1"

//Retour sur la branche master
$ git switch master
//(modification de fichiers, ajout de code, etc…)
$ git add .
$ git commit  -m "Modification de la branche principale"

$ git rebase master

//Retour sur la branche feature et ajout de commits
```

Tous droits réservés à STUDI - Reproduction interdite

```
$ git switch feature2
$ git add .
$ git commit -m "Ajout d'un deuxième commit à la branche feature"
$ git add .
$ git commit -m "Ajout d'un troisième commit à la branche feature"

$ git rebase master
$ git add .
$ git rebase –continue
```

Enfin, utilisez Git Graph pour vérifier que la fusion de la branche « *feature2* » sur la branche « *master* » s'est déroulée comme prévu.

```
* 9350a08 (HEAD -> master) Merge branch 'feature2'
* 3d8f851 (feature2) Ajout d'un troisième commit à la branche feature2
* 034f9d9 Ajout d'un deuxième commit à la branche feature2
* 1d44d19 Ajout de la nouvelle fonctionnalité
* 889a7d6 Modification de la branche principale
* 9cf4b2e Initial commit
```

Solution n°5

[exercice p. 13]

Question 1

Comment réaliser une opération de git merge ?

- Avec la commande « *git clone* »
- ✓ Avec la commande « *git merge* »
- Avec la commande « *git add -A* »

Pour réaliser une opération de git merge, vous devez fournir le nom de la branche à fusionner à la commande « *git merge* ».

Question 2

Quelle commande convient le mieux si vous travaillez sur une branche locale et que vous voulez mettre à jour votre code avec les dernières modifications de la branche principale ?

- ✓ Git merge
- Git rebase
- Les deux

Si vous travaillez dans cet environnement et ces conditions, git merge est la meilleure option, car il préserve l'historique de la branche fusionnée.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Quelle commande convient le mieux si vous travaillez sur une branche partagée avec d'autres développeurs pour éviter un historique trop complexe ?

Git merge

✓ Git rebase

Les deux

Dans le cas de figure où vous travaillez sur une branche partagée avec d'autres développeurs, git rebase est la meilleure option, car il permet d'obtenir un historique plus linéaire et facile à lire.

Question 4

Que représente le SHA dans Git ?

L'identifiant unique d'une branche Git

✓ L'identifiant unique d'un commit, d'une branche, d'une étiquette et d'une arborescence de Git

L'historique des commits dans Git

SHA est l'acronyme de *Secure Hash Algorithm*. Dans Git, le SHA est utilisé pour identifier de manière unique un commit, une branche, une étiquette ou une arborescence de Git.

Question 5

Comment peut-on trouver le SHA d'une branche Git ?

En utilisant la commande « *git log* »

✓ En utilisant la commande « *git rev-parse* » suivie du nom de la branche

En utilisant la commande « *git branch* »

La commande git rev-parse suivie du nom de la branche permet de récupérer l'identifiant unique (SHA) du commit le plus récent de la branche spécifiée.

Tous droits réservés à STUDI - Reproduction interdite